

编程题 1：返回数组中的第 k 大整数

一、算法思路

本题使用快速选择算法 QuickSelect。

快速选择的核心思想是：

每次选取一个基准值 pivot，通过 partition 操作把数组分成两部分：

左边：大于等于 pivot 的元素

右边：小于 pivot 的元素

这样 pivot 会被放到它在从大到小排列中应该处于的位置。

如果该位置正好是 $k - 1$ ，说明它就是第 k 大元素；否则只需要继续在左半部分或右半部分查找，不需要对整个数组排序。

二、算法核心代码

```
int partition(vector<int>& nums, int left, int right) {
    int pivot = nums[right];
    int i = left;

    for (int j = left; j < right; j++) {
        if (nums[j] >= pivot) {
            swap(nums[i], nums[j]);
            i++;
        }
    }

    swap(nums[i], nums[right]);
    return i;
}
```

三、核心代码解释

pivot 表示本轮选择的基准值，我们暂时取当前区间最右边的元素。变量 i 表示下一个大于等于 pivot 的元素应该放置的位置。循环过程中，如果发现： $\text{nums}[j] \geq \text{pivot}$ ，说明当前元素应该被放到左边，于是将 $\text{nums}[j]$ 和 $\text{nums}[i]$ 交换，并让 i 后移。循环结束后，再把 pivot 放到位置 i，此时 i 就是 pivot 的最终位置。

由于本题要求第 k 大，所以采用从大到小的分区方式，因此条件是 $\geq \text{pivot}$ 。

```
int quickSelect(vector<int>& nums, int left, int right, int k) {
    while (left <= right) {
        int pos = partition(nums, left, right);

        if (pos == k - 1) {
            return nums[pos];
        }
        else if (pos > k - 1) {
            right = pos - 1;
        }
    }
}
```

```

    }
    else {
        left = pos + 1;
    }
}

return -1;
}

```

partition 返回的 pos 表示基准值最终所在的位置。因为数组下标从 0 开始，所以第 k 大元素对应的下标是：k - 1

如果：

pos == k - 1

说明已经找到答案。

如果：

pos > k - 1

说明目标元素在左半部分，继续搜索左边。

如果：

pos < k - 1

说明目标元素在右半部分，继续搜索右边。

四、复杂度分析

快速选择算法平均时间复杂度为：

$O(n)$

最坏情况下时间复杂度为：

$O(n^2)$

空间复杂度为：

$O(1)$

本程序没有对数组进行预先排序，符合题目要求。核心实现见已上传代码文件。

五、测试用例

（输入前两行输出第三行）



```
Microsoft Visual Studio 调试控 × +
3 2
7 6 2
6
```

```
Microsoft Visual Studio 调试控 × +
7 5
0 5 3 3 1 6 4 2
3
```

编程题 2：寻找两个正序数组的中位数

一、算法思路

本题算法采用二分法。

核心思想是：

在两个有序数组中分别切一刀，把整体分成左半部分和右半部分，并满足：左半部分的所有元素 $\leq$ 右半部分的所有元素。如果能够找到这样的切分位置，那么中位数就可以直接由左右两边的边界元素得到。

二、算法核心代码

```
double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
    int m = nums1.size();
    int n = nums2.size();

    if (m > n) {
        return findMedianSortedArrays(nums2, nums1);
    }

    int left = 0, right = m;
    int totalLeft = (m + n + 1) / 2;

    while (left <= right) {
        int i = (left + right) / 2;
        int j = totalLeft - i;

        int nums1Left = (i == 0) ? INT_MIN : nums1[i - 1];
        int nums1Right = (i == m) ? INT_MAX : nums1[i];
```

```

int nums2Left = (j == 0) ? INT_MIN : nums2[j - 1];
int nums2Right = (j == n) ? INT_MAX : nums2[j];

if (nums1Left <= nums2Right && nums2Left <= nums1Right) {
    if ((m + n) % 2 == 1) {
        return max(nums1Left, nums2Left);
    }
    else {
        return (max(nums1Left, nums2Left) + min(nums1Right, nums2Right)) / 2.0;
    }
}

else if (nums1Left > nums2Right) {
    right = i - 1;
}

else {
    left = i + 1;
}

}

return 0.0;
}

```

三、核心代码解释

首先保证 nums1 是较短的数组：

```

if (m > n) {
    return findMedianSortedArrays(nums2, nums1);
}

```

保证二分查找的范围更小，时间复杂度为： $O(\log(\min(m, n)))$

变量： $totalLeft = (m + n + 1) / 2$; 表示左半部分一共需要放多少个元素。这样同时处理总长度为奇数和偶数的情况。

在循环中：

```

int i = (left + right) / 2;
int j = totalLeft - i;

```

i 表示在 nums1 中切分的位置，j 表示在 nums2 中切分的位置。

然后分别取出切分边界的四个值：

```

nums1Left
nums1Right
nums2Left
nums2Right

```

分别表示：

```

nums1 左半部分最大值
nums1 右半部分最小值
nums2 左半部分最大值
nums2 右半部分最小值

```

比如：

```
nums1: 1 2 |  
nums2: | 3 4
```

那么：

```
nums1Left = 2  
nums1Right = 正无穷，因为 nums1 右边没数了  
nums2Left = 负无穷，因为 nums2 左边没数  
nums2Right = 3
```

只要满足：

```
nums1Left <= nums2Right  
nums2Left <= nums1Right
```

说明切分正确。如果总长度是奇数：`return max(nums1Left, nums2Left)`；中位数就是左半部分的最大值。如果总长度是偶数：`return (max(nums1Left, nums2Left) + min(nums1Right, nums2Right)) / 2.0`；中位数是左半部分最大值和右半部分最小值的平均数。

如果：

```
nums1Left > nums2Right
```

说明 `nums1` 左边取多了，需要往左缩小范围。

否则说明 `nums1` 左边取少了，需要往右扩大范围。

六、测试用例

（输入前三行输出第四行）



```
Microsoft Visual Studio 调试控 ×  
  
2 2  
1 2  
3 4  
2.5000
```



```
Microsoft Visual Studio 调试控 × + ▾  
  
6 4  
0 0 1 3 5 6  
1 4 7 9  
3.5000
```

```
Microsoft Visual Studio 调试  ×  +  
2 5  
1 3  
0 2 4 5 6  
3.0000
```